UNIVERSIDAD NACIONAL DEL SANTA

FACULTAD DE INGENIERÍA

ESCUELA ACADÉMICO PROFESIONAL DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

PRACTICA S05

CURSO:
AGENTES INTELIGENTES

DOCENTE:
Ms. Borja Reyna Whiston Kendrick

ESTUDIANTES

Álvarez Paredes José Luis – 0202314003
Mendoza Chavez Raul Andre - 0202314036

NUEVO CHIMBOTE – PERU
2026

Descripción de los Códigos

Código 1: Agente Deliberativo BDI

Vigila una plataforma de comercio electrónico y decide cómo actuar ante fallas. La arquitectura BDI divide el razonamiento en tres partes: las creencias, que son lo que el agente sabe del entorno; los deseos, que son las metas posibles; y las intenciones, que son las metas que decide cumplir (Rao & Georgeff, 1995; Wooldridge, 2009). En cada ciclo, el agente recibe datos del clúster, actualiza sus creencias, genera deseos, elige una intención y ejecuta un plan.

Código 2: Q-Learning con Reward Shaping

El archivo practica_semana04_rl.py simula un dron que debe llegar a una meta en una cuadrícula de 4x4 con viento aleatorio. El dron no conoce el mapa y aprende por prueba y error con Q-Learning, un método de aprendizaje por refuerzo que guarda en una tabla el valor esperado de cada acción en cada posición (Sutton & Barto, 2018; Watkins & Dayan, 1992). El entorno es estocástico: la acción elegida se cumple solo en un tercio de los casos y, en los demás, el dron se desvía hacia un lado.

Objetivos

El objetivo del primer código es lograr que el agente detecte los problemas a tiempo, atienda primero el más importante y cambie de plan, sin perder su meta, cuando un plan deja de ser posible. En concreto, debe deducir una fuga de memoria a partir de tres lecturas de CPU, aplicar las prioridades indicadas y usar un plan alternativo cuando se agota la cuota de la nube.

El objetivo del segundo código es que el agente aprenda una forma de llegar a la meta y que se comparen dos esquemas de recompensa. Se busca aplicar la fórmula del error de diferencia temporal, usar la exploración ε-greedy con decaimiento y medir la tasa de éxito por bloques de episodios.

Elementos Añadidos

Código 1

Además de completar las partes marcadas como pendientes, se añadió lo siguiente. Primero, constantes para los umbrales (80 % de CPU, 15 puntos por ciclo y 500 ms de latencia), que permiten ajustarlos con facilidad. Segundo, las creencias cpu_actual y latencia_ms, necesarias para generar los deseos. Tercero, la función ejecutar_ciclo, que une todos los pasos del agente, y la función construir_biblioteca, que define los planes. Cuarto, tres planes adicionales (seguridad, caché y fuga de memoria), para que el agente tenga una acción en cada situación. Quinto, tres escenarios de prueba. No se precisa qué condición activa los deseos de caché y de rendimiento, por lo que se colocó que los activan la CPU crítica y la latencia alta, respectivamente.

Código 2

Se hicieron varios cambios para mejorar el experimento. Se corrigió la estocasticidad, pues la plantilla daba 34 % de éxito en la acción deseada y ahora se logra un tercio exacto. Se agregó un desempate aleatorio cuando varias acciones tienen el mismo valor, ya que al inicio la tabla está en ceros y el agente repetiría siempre la misma acción. Se añadió un límite de 100 pasos por episodio para evitar episodios sin fin. Se redujo la tasa de aprendizaje de 0,8 a 0,1, porque un valor alto hace que el agente olvide rápido lo aprendido en un entorno tan variable. También se evitó sumar valor futuro en los estados finales. Por último, se incluyeron funciones para medir la tasa de éxito por bloques, evaluar la política final, mostrar la política aprendida y repetir el experimento con cinco semillas aleatorias.

Funcionamiento

Código 1

La función brf revisa las creencias. Guarda las últimas tres lecturas de CPU y, si aumentan en cada ciclo y el incremento medio es de al menos 15 puntos, deduce que existe una fuga de memoria. Esto ocurre con la serie 40, 58 y 75, aunque la CPU no llegue al 80 %. La función options crea los deseos según las creencias, con prioridad 100 para la seguridad, 80 para la caché, 75 para la memoria y 60 para el rendimiento. La función filter elige el deseo de mayor prioridad y, como se ejecuta en cada ciclo, permite que el agente cambie de meta si aparece algo más urgente.

La función seleccionar_plan_fallback busca en la biblioteca el primer plan que sirva para la meta, cumpla su condición y no esté marcado como fallido. Finalmente, ejecutar_ciclo realiza una acción por ciclo. Si el plan activo deja de ser posible, lo marca como fallido, conserva la intención y elige otro plan. En la prueba, al agotarse la cuota, el agente pasó del plan de autoescalado al plan de *Circuit Breaker* y no volvió a intentar el primero.

Código 2

El entorno recibe una acción y aplica el viento: con probabilidad de dos tercios, la acción se desvía hacia un lado. Luego mueve al dron y entrega la recompensa. En el esquema estándar solo se recibe +1 al llegar a la meta. En el esquema con *reward shaping* se resta 0,01 por cada paso, se resta 1 al caer en un hoyo y se suma 1 en la meta.

La función entrenar_q_learning repite episodios. En cada paso elige una acción con la política ε-greedy: con probabilidad ε explora al azar y, en otro caso, elige la mejor acción conocida. El valor de ε empieza en 1,0, se multiplica por 0,995 en cada episodio y no baja de 0,01. Después de actuar, el agente actualiza su tabla con la regla de Q-Learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [\, r + \gamma \cdot \max Q(s', a') - Q(s, a) \,]$$

Es decir, calcula un objetivo (la recompensa más el mejor valor del siguiente estado, reducido por γ), obtiene el error al restar el valor actual y corrige el valor en una fracción α de ese error.

Resultados

La Tabla 1 muestra el promedio de cinco ejecuciones. El *reward shaping* permitió aprender más rápido y alcanzar una mejor política. Esto se explica porque, sin penalizaciones, el agente recibe información solo al llegar a la meta, algo poco frecuente al inicio. Con *reward shaping* recibe una señal en cada paso, aprende a evitar los hoyos y prefiere rutas cortas.

Tabla 1

Tasa de Éxito del Agente según el Esquema de Recompensa

Indicador	Entorno estándar	Reward shaping
Éxito en episodios 301–600	11,6 %	32,4 %
Éxito en episodios 601–900	27,4 %	59,2 %
Éxito de la política final	57,5 %	70,5 %

Nota. Promedio de cinco semillas aleatorias con 3000 episodios de entrenamiento. La política final se evaluó en 2000 episodios del entorno estándar.

Conclusiones

Los dos códigos muestran dos formas de construir agentes. El agente BDI decide con reglas claras, prioridades y planes de respaldo, por lo que su comportamiento es fácil de explicar. El agente de Q-Learning no recibe reglas, sino que aprende de la experiencia, y su aprendizaje mejora cuando las recompensas están bien diseñadas.

Referencias Bibliograficas

Rao, A. S., & Georgeff, M. P. (1995). BDI agents: From theory to practice. En *Proceedings of the First International Conference on Multi-Agent Systems* (pp. 312–319). AAAI Press.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2.ª ed.). MIT Press.

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning, 8*(3–4), 279–292. https://doi.org/10.1007/BF00992698

Wooldridge, M. (2009). *An introduction to multiagent systems* (2.ª ed.). John Wiley & Sons.